

Tutorial — Usando Regex para representar e validar um e-mail

Linguagens Formais e Autômatos — Material do estudante

Professora Kadidja Valéria

1. Objetivo

Neste tutorial, você aprenderá a construir uma expressão regular para verificar se um texto apresenta o formato básico de um endereço de e-mail.

Importante: uma Regex não escreve o conteúdo de uma mensagem. Ela reconhece, pesquisa ou valida padrões de texto, como `nome@dominio.com`.

2. Estrutura de um endereço de e-mail

Um e-mail geralmente possui três partes:

```
usuario@dominio.extensao
```

Exemplo:

```
kadidja.oliveira@udf.edu.br
```

Parte	Exemplo	Descrição
Usuário	kadidja.oliveira	Identifica a pessoa ou conta
Separador	@	Separa o usuário do domínio
Domínio	udf.edu.br	Identifica o serviço ou instituição

3. Construção progressiva da Regex

Etapas 1 — Reconhecer o nome do usuário

Podemos começar aceitando letras, números, pontos, sublinhados, sinais de adição e hífen:

[A-Za-z0-9._+-]+

Significado:

- [A-Za-z0-9._+-]: aceita um caractere pertencente ao conjunto;
- A-Z: letras maiúsculas;
- a-z: letras minúsculas;
- 0-9: algarismos;
- .: ponto;
- _: sublinhado;
- +: sinal de adição;
- -: hífen;
- + depois dos colchetes: exige um ou mais caracteres.

Essa parte reconhece exemplos como:

aluno
kadidja.oliveira
estudante_01
contato+curso

Etapas 2 — Acrescentar o símbolo @

[A-Za-z0-9._+-]+@

O símbolo @ aparece literalmente porque é obrigatório no endereço.

Etapas 3 — Reconhecer o domínio

[A-Za-z0-9.-]+

Essa parte aceita letras, números, pontos e hífen.

Exemplos:

gmail
udf.edu
minha-instituicao

Etapas 4 — Reconhecer a extensão

\.[A-Za-z]{2,}

Significado:

- \.: representa um ponto literal;
- [A-Za-z]: aceita uma letra;
- {2,}: exige pelo menos duas letras.

Exemplos reconhecidos:

```
.com
.br
.edu
.info
```

4. Expressão regular completa

Reunindo todas as partes:

```
^[A-Za-z0-9._+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$
```

A estrutura pode ser visualizada assim:

```
^[A-Za-z0-9._+-]+  @  [A-Za-z0-9.-]+  \.  [A-Za-z]{2,}$
      usuário           domínio           extensão
```

Função dos delimitadores

- ^: indica o início do texto;
- \$: indica o final do texto.

Esses símbolos impedem que a Regex aceite um e-mail válido apenas como parte de um texto maior.

5. Exemplos de teste

Entrada	Resultado esperado	Motivo
aluno@gmail.com	Válido	Possui usuário, @, domínio e extensão
nome.sobrenome@udf.edu.br	Válido	Aceita pontos e mais de uma parte no domínio
estudante_01@faculdade.com	Válido	Aceita sublinhado e números
contato+curso@exemplo.com.br	Válido	Aceita o sinal +
aluno.gmail.com	Inválido	Não possui @
aluno@	Inválido	Não possui domínio
@gmail.com	Inválido	Não possui usuário
aluno@gmail	Inválido	Não possui extensão
aluno@gmail.c	Inválido	A extensão possui somente uma letra
aluno gmail.com	Inválido	Contém espaço e não possui @

6. Testando a Regex em Python

```
import re

padrao_email = r"^[A-Za-z0-9._+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$"

email = input("Digite um endereço de e-mail: ")

if re.fullmatch(padrao_email, email):
    print("O e-mail apresenta um formato válido.")
else:
    print("O formato do e-mail é inválido.")
```

Como o código funciona

1. `import re` carrega o módulo de expressões regulares do Python.
2. `padrao_email` armazena a Regex.
3. `input()` solicita um endereço ao usuário.
4. `re.fullmatch()` verifica se todo o texto corresponde ao padrão.
5. A estrutura `if` informa se o formato foi aceito.

Uma alternativa mais legível é retirar `^` e `$`, pois `fullmatch()` já exige a correspondência do texto completo:

```
import re

padrao_email = r"[A-Za-z0-9._+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}"

emails = [
    "aluno@gmail.com",
    "nome.sobrenome@udf.edu.br",
    "aluno.gmail.com",
    "aluno@gmail"
]

for email in emails:
    if re.fullmatch(padrao_email, email):
        print(f"{email}: formato válido")
    else:
        print(f"{email}: formato inválido")
```

7. Utilização em um formulário HTML

O atributo `pattern` permite aplicar a Regex a um campo de formulário:

```
<form>
  <label for="email">E-mail:</label>

  <input
    type="email"
    id="email"
    name="email"
    pattern="[A-Za-z0-9._+- ]+@[A-Za-z0-9.- ]+\.[A-Za-z]{2,}"
    required
  >

  <button type="submit">Enviar</button>
</form>
```

O navegador verifica o formato antes de enviar o formulário. Entretanto, a aplicação também deve validar os dados no servidor.

8. Relação com Linguagens Formais

A Regex descreve uma linguagem formada por cadeias que seguem determinado padrão.

Podemos representar, de maneira simplificada:

E-MAIL = USUÁRIO · @ · DOMÍNIO · . · EXTENSÃO

Em que:

- USUÁRIO contém um ou mais caracteres permitidos;
- @ é um símbolo obrigatório;
- DOMÍNIO contém um ou mais caracteres permitidos;
- . é um símbolo literal obrigatório;
- EXTENSÃO contém pelo menos duas letras.

Assim, a expressão regular funciona como uma regra para decidir se uma palavra pertence à linguagem dos endereços de e-mail aceitos pelo sistema.

9. Limitações importantes

A Regex apresentada realiza uma validação didática e simplificada. Ela verifica o formato, mas não garante que:

- o endereço realmente exista;
- o domínio esteja registrado;
- a caixa de entrada esteja ativa;
- o usuário tenha acesso ao endereço;
- todas as regras técnicas internacionais de e-mail sejam atendidas.

A confirmação completa normalmente exige o envio de uma mensagem com link ou código de verificação.

10. Atividade prática

Construa um programa que:

1. solicite cinco endereços de e-mail;
2. valide cada endereço com uma Regex;
3. armazene separadamente os formatos válidos e inválidos;
4. apresente os dois grupos ao final;
5. explique por que cada entrada inválida foi rejeitada.

Utilize pelo menos estes casos de teste:

```
maria@gmail.com  
joao.silva@udf.edu.br  
estudante_01@faculdade.com  
pedro@gmail.com  
ana@dominio
```

Desafio

Aprimore a Regex para impedir:

- ponto no início do usuário;
- ponto imediatamente antes de @;
- dois pontos consecutivos;
- hífen no início ou no final de uma parte do domínio.

Em aplicações reais, evite criar uma Regex excessivamente rígida: endereços válidos podem ter formatos menos comuns.